

```
#!/bin/bash
```

```
# Comprobar el número correcto de argumentos
```

```
if [ "$#" -ne 1 ]; then  
    echo "Uso: $0 <directorío>"  
    exit 1  
fi
```

```
directorío="$1"
```

```
# Comprobar si el directorío existe
```

```
if [ ! -d "$directorío" ]; then  
    echo "El directorío $directorío no existe."  
    exit 1  
fi
```

```
# Función para obtener la fecha de modificación en segundos desde la época
```

```
function obtener_fecha_modificacion {  
    local archivo="$1"  
    local fecha_modificacion=$(stat -c %Y "$archivo")  
    # El comando stat se utiliza para mostrar información sobre el  
    # archivo especificado, y -c %Y se usa para especificar el formato de  
    # salida deseado, que en este caso es la fecha de modificación en  
    # segundos desde la época. $archivo es la variable que contiene la  
    # ruta completa del archivo del cual se desea obtener la fecha de  
    # modificación.  
    echo "$fecha_modificacion"  
}
```

```
# Función para obtener el tamaño en bytes del archivo
```

```
function obtener_tamano {  
    local archivo="$1"  
    local tamano=$(stat -c %s "$archivo")  
    # -c %s: se usa para especificar el formato de salida deseado, que en  
    # este caso es el tamaño del archivo en bytes  
    echo "$tamano"  
}
```

```
# Función para obtener la cadena de permisos del archivo
```

```
function obtener_permisos {  
    local archivo="$1"  
    local permisos=$(stat -c %A "$archivo")  
    # -c %A: se utiliza para especificar el formato de salida deseado, que  
    # en este caso es la cadena de permisos en formato legible por humanos.  
    echo "$permisos"  
}
```

```

# Obtener la lista de archivos (sin los subdirectorios) y ordenarlos por fecha de modificación
archivos=$(find "$directorio" -type f -printf "%ft%p\t" -exec stat -c %Y:%s:%A {} \; | sort -n -t: -k1 | cut -f 2-)
# f: fichero
# Con la opción -printf, find formatea la salida para imprimir el nombre
# del archivo (%f), seguido de un tabulador (\t), luego la ruta completa
# del archivo (%p), y otro tabulador al final. Esto asegura que cada
# línea de salida contenga el nombre del archivo, la ruta completa y un
# delimitador de tabulación entre ellos.
# -exec: para cada archivo encontrado ejecuta el comando a continuacion
# | sort -n -t: -k1: La salida de find y stat se pasa a sort, que ordena
# las líneas de entrada numéricamente (-n) en función del primer campo
# (-k1), utilizando : como delimitador de campo (-t:). Esto ordena los
# archivos por fecha de modificación en orden ascendente.
# | cut -f 2-: Finalmente, se utiliza cut para recortar (-f) los campos de
# la salida de sort, desde el segundo campo hasta el final. Esto elimina
# el nombre del archivo y deja solo la ruta completa, la fecha de
# modificación, el tamaño del archivo y los permisos del archivo, todos
# ellos separados por :

# Mostrar la información de cada archivo
while IFS=$'\t' read -r nombre_archivo ruta_archivo fecha_modificacion tamano permisos; do
    echo -e "$nombre_archivo\t$ruta_archivo\t$fecha_modificacion\t$tamano bytes\t$permisos"
done <<< "$archivos"
# IFS=$'\t': Establece el separador de campo interno (IFS) como un
# tabulador ('\t'). Esto indica a read que utilice el tabulador como
# delimitador de campos al leer las líneas de entrada, de modo que cada
# campo se almacene correctamente en las variables.
# read -r: Lee una línea del texto de entrada y divide los campos
# separados por tabuladores en las variables indicadas
# (nombre_archivo, ruta_archivo, fecha_modificacion, tamano y permisos)
# echo -e: Imprime en la salida estándar una línea formateada que contiene
# la información de cada archivo. Cada campo está separado por un
# tabulador \t, y el tamaño del archivo está seguido por la cadena "bytes"
# <<< "$archivos": Utiliza la redirección para pasar el contenido de la
# variable $archivos como entrada al bucle while.

```